

Process & Decision Documentation

Week 9 Side Quests:

Focus on:

- Building a developer debug screen with toggleable features (No-Clip, Force Game Over)
- Implementing a state machine (Start, Play, Game Over, Win)
- Adding procedural obstacle generation to a grid-based maze game.

GenAI Documentation

If GenAI was used (keep each response as brief as possible):

Date Used: March 23, 2026

Tool Disclosure: Gemini Pro

Purpose of Use: Structuring the state machine logic, designing the floating HUD debug menu, generating random obstacles, and mapping out the 2D arrays for the 5-stage levels.json file.

Summary of Interaction: The AI provided the initial logic for the debug menu and state machine. I then directed the AI to modify the debug menu into a floating HUD and requested the addition of procedural logic to randomly spawn walls on empty tiles for increased replayability.

Human Decision Point(s): I rejected the AI's initial suggestion to automatically resize the canvas. Because of this, I decided to shrink the debug menu into a floating UI element so it wouldn't block the gameplay. I also designed the layout of the on-screen instructions, ensuring the debug screen was clearly visible below the standard movement controls.

Integrity & Verification Note: I verified the code by running it locally in VS Code. I tested the debug menu toggles. I also playtested the full game loop from the Start screen to the Win screen to confirm the gameState logic.

Scope of GenAI Use: The AI generated the core logic for the state machine, procedural generation, and debug HUD overlay. I guided the UX/UI design choices, defined the layout for the on-screen instructions, controlled the canvas dimensions, and dictated the overall flow of the game states.

Limitations or Misfires: Initially, the AI generated a full-screen debug menu that blocked the gameplay, so I had to explicitly direct it to create a smaller floating HUD. Additionally, during one of the UI updates, the AI accidentally removed the trigger that drew the debug menu; I caught this during my VS Code testing and instructed the AI to fix the missing code block.

Appendix

hi please read through the instruction for side quest, this is the side quest for this week:

If you completed Side Quest 6, then take your completed Side Quest 6 and add a debug screen. If you didn't do Side Quest 6, add a debug screen to Week 9 - Example 3 that allows you to toggle moon gravity on/off, and at least one other potential feature you might need in developing a game.

Bonus: If you completed Side Quest 6, add another level to it. If not, then add another level to Example 4 complete with sound.

start with the basic one, do not do the bonus yet. I've gave you the side quest 6

Gemini said

Hey Fiona! It is great to see your progress in GBDA 302. Your Week 6 sketch with the `lerp()` physics and particle system is looking incredibly polished!

Since your Side Quest 6 game is a top-down maze, adding "moon gravity" doesn't really fit your mechanics. Instead, we will build a debug screen tailored to your specific game design, complete with features you would actually use when testing a grid-based puzzle game.

Here is the plan for your baseline Week 9 requirement:

1. **Toggleable Overlay:** A semi-transparent screen that opens when you press the **D** key.
2. **"No-Clip" Mode:** A classic developer cheat. By pressing **N** while in debug mode, you can toggle the ability to walk straight through walls to test pathing.
3. **Level Skipper:** Pressing **L** will instantly jump you to the next level.
4. **Live Stats:** Real-time tracking of your player's (*x*, *y*) coordinates and the game's Frames Per Second (FPS).

Updated `sketch.js`

Here is your updated engine. I have clearly marked the new sections with `// --- NEW: Week 9 Debug Screen ---` so you can easily see what was added to your existing logic.

JavaScript

```
let levelData;
```


